

COMPLETE TECH STACK

Programming Language

- Python

Libraries

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Streamlit

Database

- SQLite

Machine Learning

- Logistic Regression
- Random Forest
- Isolation Forest

Visualization

- Power BI OR Streamlit Dashboard

PROJECT ARCHITECTURE

Transaction Data ↓ Data Cleaning ↓ EDA (Exploratory Data Analysis) ↓ SQL Analytics ↓
Feature Engineering ↓ Machine Learning Model ↓ Fraud Prediction ↓ Dashboard
Visualization ↓ Risk Analytics

STEP 1 — DOWNLOAD DATASET

Dataset

Use this dataset:

Credit Card Fraud Detection Dataset

Dataset Link: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

ABOUT THE DATASET

The dataset contains:

- Transaction amount
- Transaction time
- Features related to transaction behavior
- Fraud label

Important Column

Class Column

This is the target column.

- 0 = Normal transaction
- 1 = Fraudulent transaction

This means:

Your ML model must predict whether a transaction is fraud or not.

STEP 2 — CREATE PROJECT FOLDER

Create Folder Structure

Create:

```
financial_fraud_detection/ | |— data/ |— notebooks/ |— sql/ |— dashboard/ |—  
models/ |— screenshots/ |— app.py |— requirements.txt |— README.md
```

EXPLANATION OF EACH FOLDER

data/

Store dataset files.

notebooks/

Store Jupyter notebooks.

sql/

Store SQL queries.

dashboard/

Dashboard-related files.

models/

Saved ML models.

screenshots/

Dashboard screenshots.

app.py

Main Streamlit application.

requirements.txt

Contains all required Python libraries.

README.md

Project explanation for GitHub.

STEP 3 — INSTALL REQUIRED LIBRARIES

Command

```
pip install pandas numpy matplotlib seaborn scikit-learn streamlit joblib
```

EXPLANATION OF EACH LIBRARY

pandas

Used for:

- reading CSV files
- data analysis
- cleaning data

numpy

Used for:

- numerical operations
- arrays
- mathematical calculations

matplotlib

Used for:

- plotting graphs
- visualizations

seaborn

Used for:

- advanced visualizations
- heatmaps
- distributions

scikit-learn

Used for:

- machine learning models
- preprocessing
- evaluation metrics

streamlit

Used for:

- building web dashboard

joblib

Used for:

- saving ML model
-

STEP 4 — LOAD DATASET

Create File

Inside notebooks/ create:

fraud_analysis.ipynb

CODE

```
import pandas as pd

# Load dataset
df = pd.read_csv('data/creditcard.csv')

# Display first 5 rows
print(df.head())
```

LINE-BY-LINE EXPLANATION

import pandas as pd

Imports pandas library.

pd is a shortcut name.

Instead of writing:

pandas.read_csv()

we can write:

pd.read_csv()

```
df = pd.read_csv('data/creditcard.csv')
```

This reads the CSV dataset.

- `read_csv()` → function to read CSV file
- `df` → dataframe variable

DataFrame means: A table-like structure containing rows and columns.

```
print(df.head())
```

`head()` displays first 5 rows.

Purpose:

- verify dataset loaded correctly
 - understand column structure
-

STEP 5 — UNDERSTAND DATASET

CODE

```
print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())
```

EXPLANATION

df.shape

Returns: (number_of_rows, number_of_columns)

Example:

(284807, 31)

Meaning:

- 284807 rows
- 31 columns

df.columns

Displays all column names.

Useful for:

- understanding features
- selecting variables

df.info()

Shows:

- column names
- datatype
- null values

Important for data cleaning.

df.describe()

Shows statistical summary:

- mean
- min
- max
- standard deviation

Used for understanding data distribution.

STEP 6 — CHECK MISSING VALUES

CODE

```
print(df.isnull().sum())
```

EXPLANATION

isnull()

Checks whether value is missing.

Returns:

- True
 - False
-

sum()

Counts total missing values.

If output is 0: Dataset has no missing values.

STEP 7 — CHECK FRAUD DISTRIBUTION

CODE

```
print(df['Class'].value_counts())
```

EXPLANATION

df['Class']

Selects Class column.

value_counts()

Counts frequency of each category.

Example:

```
0 -> 284315  
1 -> 492
```

Meaning:

- Most transactions are normal
- Very few are fraud

This is called:

Imbalanced Dataset

Very important concept in fraud analytics.

STEP 8 — VISUALIZE FRAUD DISTRIBUTION

CODE

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.countplot(x='Class', data=df)
plt.title('Fraud vs Normal Transactions')
plt.show()
```

LINE-BY-LINE EXPLANATION

import matplotlib.pyplot as plt

Imports plotting module.

import seaborn as sns

Imports seaborn visualization library.

sns.countplot(x='Class', data=df)

Creates bar chart.

x='Class' means: plot Class column categories.

plt.title()

Adds graph title.

plt.show()

Displays graph.

STEP 9 — EXPLORATORY DATA ANALYSIS (EDA)

Goal

Understand:

- patterns
 - anomalies
 - trends
 - relationships
-

Analyze Transaction Amount

CODE

```
plt.figure(figsize=(10,5))
sns.histplot(df['Amount'], bins=50)
plt.title("Transaction Amount Distribution")
plt.show()
```

EXPLANATION

figsize=(10,5)

Sets graph size.

histplot()

Creates histogram.

Histogram shows: How values are distributed.

bins=50

Divides graph into 50 sections.

STEP 10 — CORRELATION ANALYSIS

CODE

```
corr = df.corr()

plt.figure(figsize=(12,10))
sns.heatmap(corr)
plt.title('Correlation Matrix')
plt.show()
```

EXPLANATION

df.corr()

Calculates correlation between columns.

Correlation means: How strongly variables are related.

sns.heatmap()

Displays correlation visually.

Useful for:

- identifying strong relationships
 - feature selection
-

STEP 11 — FEATURE & TARGET SPLIT

CODE

```
X = df.drop('Class', axis=1)  
y = df['Class']
```

EXPLANATION

X

Contains input features.

y

Contains target variable.

Target: What model must predict.

drop('Class', axis=1)

Removes Class column.

axis=1 means: remove column.

STEP 12 — TRAIN TEST SPLIT

CODE

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
y,  
test_size=0.2,  
random_state=42  
)
```

EXPLANATION

train_test_split()

Splits dataset into:

- training data
 - testing data
-

test_size=0.2

20% data used for testing.

80% used for training.

random_state=42

Ensures reproducible results.

STEP 13 — TRAIN MACHINE LEARNING MODEL

Logistic Regression

CODE

```
from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression()  
model.fit(X_train, y_train)
```

EXPLANATION

LogisticRegression()

Creates classification model.

Used for:

- binary classification
 - yes/no prediction
-

model.fit()

Trains model.

Model learns patterns from training data.

STEP 14 — MAKE PREDICTIONS

CODE

```
predictions = model.predict(X_test)
```

EXPLANATION

predict()

Uses trained model to predict fraud.

STEP 15 — EVALUATE MODEL

CODE

```
from sklearn.metrics import classification_report  
  
print(classification_report(y_test, predictions))
```

EXPLANATION

classification_report shows:

- precision
- recall
- F1-score
- accuracy

These metrics are VERY important in analytics interviews.

STEP 16 — RANDOM FOREST MODEL

CODE

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier()  
rf_model.fit(X_train, y_train)
```

WHY RANDOM FOREST?

Random Forest:

- handles complex patterns
- reduces overfitting
- works well for fraud detection

This is commonly used in industry.

STEP 17 — ANOMALY DETECTION

CODE

```
from sklearn.ensemble import IsolationForest
```

```
iso_model = IsolationForest(contamination=0.002)  
iso_model.fit(X)
```

EXPLANATION

Isolation Forest detects:

- unusual behavior
- anomalies
- suspicious transactions

Very important in:

- banking
 - cybersecurity
 - fraud analytics
-

STEP 18 — SAVE MODEL

CODE

```
import joblib

joblib.dump(rf_model, 'models/fraud_model.pkl')
```

EXPLANATION

dump()

Saves trained model.

.pkl means: serialized machine learning model.

STEP 19 — BUILD STREAMLIT DASHBOARD

Create **app.py**

CODE

```
import streamlit as st
import pandas as pd
import joblib

model = joblib.load('models/fraud_model.pkl')

st.title('Financial Fraud Detection System')

uploaded_file = st.file_uploader('Upload CSV File')

if uploaded_file:
    data = pd.read_csv(uploaded_file)

    predictions = model.predict(data)

    data['Fraud Prediction'] = predictions

    st.write(data.head())
```

LINE-BY-LINE EXPLANATION

import streamlit as st

Imports Streamlit.

Used for web dashboard.

model = joblib.load()

Loads saved ML model.

st.title()

Adds title to dashboard.

st.file_uploader()

Allows user to upload CSV.

model.predict(data)

Predicts fraud transactions.

data['Fraud Prediction']

Creates new prediction column.

STEP 20 — RUN APPLICATION

Command

```
streamlit run app.py
```

WHAT HAPPENS?

Your browser opens.

You get:

- upload button
- predictions
- fraud analytics dashboard

This becomes a real analytics application.

STEP 21 — SQL ANALYTICS (VERY IMPORTANT)

Create SQL analysis queries.

EXAMPLE QUERY 1

Fraud Count

```
SELECT Class, COUNT(*)  
FROM transactions  
GROUP BY Class;
```

EXPLANATION

GROUP BY

Groups similar categories.

Used heavily in analytics.

EXAMPLE QUERY 2

High Amount Transactions

```
SELECT *  
FROM transactions  
WHERE Amount > 1000;
```

PURPOSE

Find suspicious high-value transactions.

EXAMPLE QUERY 3

Average Transaction Amount

```
SELECT AVG(Amount)  
FROM transactions;
```

STEP 22 — ADD BUSINESS INSIGHTS

Your project should explain:

Example Insights

- Fraud mostly occurs during unusual transaction patterns
- High-value transactions carry increased fraud risk
- Certain transaction behaviors indicate anomalies

This makes project look INDUSTRY LEVEL.

STEP 23 — CREATE README FOR GITHUB

README should contain:

- Project Overview
 - Features
 - Tech Stack
 - Dataset
 - Screenshots
 - Results
 - Future Improvements
-

STEP 24 — TAKE SCREENSHOTS

Take screenshots of:

- dashboard
- graphs
- fraud prediction output
- SQL results

Very important for recruiters.

STEP 25 — DEPLOYMENT (OPTIONAL)

You can later deploy using:

- Streamlit Cloud
- Render
- Hugging Face Spaces

Then add deployed link in resume.

RESUME DESCRIPTION (FUTURE)

Developed an AI-powered financial fraud detection and risk analytics platform using Python, SQL, Random Forest, and Isolation Forest to analyze transactional anomalies and predict fraudulent activity. Built interactive dashboards and risk analytics visualizations using Streamlit to support real-time fraud monitoring and business insights.

WHAT YOU WILL LEARN FROM THIS PROJECT

By completing this project, you will master:

- Python
- SQL
- EDA
- Data Cleaning
- Feature Engineering
- Machine Learning
- Fraud Analytics
- Dashboard Development
- Streamlit
- Risk Analytics
- Business Insights
- GitHub Documentation